

Optimization Dynamics and Regularization in Neural Network Training: A Controlled Study on Adult Income and Wine Quality

Umar Rafi
urafi3@gatech.edu

Abstract

This report looks at how specific training choices actually affect neural network classifiers, rather than just chasing better numbers. Using fixed PyTorch MLP backbones on Adult Income and Wine Quality, we test three things under matched compute budgets: randomized optimization on frozen last layers, ablations of Adam optimizer components, and explicit regularization. The results are honestly mixed—RO did not consistently beat gradient-based training, and stacking regularizers on a small network made things worse, not better. But these outcomes are informative. We also correct a known label-indexing bug from the SL Wine pipeline, which turns out to matter more than any algorithmic choice tested here.

I Introduction

Our SL Report established baseline classifiers for Adult Income (binary, F1 = 0.670 for PyTorch NN) and Wine Quality (multiclass). For Wine, the SL pipeline had a confirmed bug—the PyTorch output dimension was set to 10 instead of 7, and quality labels were not remapped to contiguous 0-indexed integers as required by `CrossEntropyLoss`. The corrected Wine baseline under a valid pipeline is Macro-F1 = 0.512, which we use as the true comparison point throughout this report. The SL figure of 0.608 is not a valid reference because it was produced under misaligned class indices.

This report asks whether targeted interventions—randomized optimization, optimizer tuning, regularization—can improve on these baselines, and more importantly, *why* they do or do not work. The goal is to trace cause to effect. We control budgets strictly: every comparison uses the same number of gradient or function evaluations, the same architecture, and the same data splits. Observed differences should therefore reflect genuine causal effects rather than experimental confounds.

A secondary goal is to understand whether combining individually positive techniques gives additive gains. The experiments provide a direct test of that

assumption across three axes: the optimization algorithm, the optimizer configuration, and the regularization strategy. Each axis is isolated in turn so that outcomes can be attributed to specific choices rather than confounded combinations.

A Hypotheses

These were written before running experiments. H1 (Adult, RO): the loss surface near the SL-trained weights is relatively flat in the last-layer subspace, so RO with 5,000 evaluations will not consistently improve test F1 beyond 0.670. H2 (Wine, RO): RO will perform at or below 0.512 with high variance across seeds, since the frozen backbone leaves limited room for adjustment across seven quality classes. H3 (Adult, Adam ablations): Adam-family methods will converge faster than SGD in wall-clock time but show a wider generalization gap; removing bias correction will slow early-phase progress; AdamW should narrow the gap somewhat. H4 (Adult, Regularization): the `hidden=[100]` network is not heavily overfitting to begin with, so regularization headroom is limited—L2 and early stopping may help slightly, but aggressive dropout and label smoothing will likely hurt.

II Datasets and Preprocessing

A Adult Income

The Adult Income dataset has census data for binary classification of income exceeding \$50K/year, with roughly 24% positive examples. We reproduce the SL 60/20/20 train/val/test split exactly (stratified, `random_state=42`). The preprocessor is fit only on training data. Numerical features—age, `fnlwgt`, education-num, capital-gain, capital-loss, and hours-per-week—receive median imputation and standard scaling. Categorical features—workclass, education, marital-status, occupation, relationship, race, sex, and native-country—get constant imputation with a “missing” category and one-hot encoding, producing 103 input features. F1-score is the primary metric given class imbalance. As a sanity check, a dummy classifier predicting the majority class achieved F1 = 0.0 on the positive class, confirming no trivial solu-

tion exists.

B Wine Quality

Wine Quality combines red and white wines with physicochemical features predicting discrete quality scores. As noted above, labels are remapped to 0-indexed integers via `LabelEncoder` (quality 3→0 through 9→6) to fix the SL pipeline bug. Standard scaling is applied to all 12 features. The split mirrors Adult. The corrected baseline is Macro-F1 = 0.512; any comparison to the SL figure of 0.608 would be misleading given the confirmed index misalignment.

C EDA Consistency

No material changes were made to cleaning, feature handling, or split methodology beyond the Wine label fix. That fix is a bug correction, not a design change, and it is the main reason Wine results look different here than in the SL report.

III Experimental Setup

A Backbone Architecture

Both datasets use the compact PyTorch MLP from the SL Report. For Adult: input dimension 103, single hidden layer of 100 units, output dimension 2, trained with `CrossEntropyLoss`. For Wine: input dimension 12, hidden layer of 100 units, output dimension 7. ReLU activation follows each hidden layer with no dropout in the baseline. SL training used Adam [1] with `lr=0.01` and `wd=0`. This architecture is fixed for all experiments; only the regularization section adds dropout modules, which is documented explicitly.

B Compute Budgets

Three cost measures are tracked. Gradient evaluations count each backward pass for SGD and Adam-family methods. Function evaluations count each complete forward pass over the full validation set with `model.eval()` enforced, used only for RO. Wall-clock time is measured on an Intel i7 CPU with no GPU. RO gets a maximum of 5,000 function evaluations per algorithm. Adam ablations and regularization use 30 epochs with batch size 64, giving roughly 12,720 gradient updates on Adult. Part IV limits RO fine-tuning to 500 function evaluations. All seeds are fixed at 42, with multi-seed runs across seeds 42, 43, and 44.

IV Randomized Optimization

A Design

All layers except the last two linear layers are frozen. For Adult with `hidden=[100]` and output dimension 2, this gives 10,602 trainable parameters—well under

the 50k cap. Wine has a similar parameter count. The objective is validation cross-entropy loss over the full validation set; each such computation is one function evaluation. All algorithms warm-start from weights obtained by 20-epoch Adam pretraining.

Randomized Hill Climbing uses a Gaussian step with $\sigma = 0.01$, three restarts, and a plateau rule stopping a restart after 500 consecutive evaluations without improvement, restarting from $\mathcal{U}(-1, 1)$. Simulated Annealing uses $T_0 = 1.0$ with exponential decay $T_t = T_0 \cdot 0.995^t$ and $\sigma = 0.01$; acceptance rates are logged every 100 evaluations and confirmed to decrease over time as expected, verifying the schedule is neither too hot nor too cold. The Genetic Algorithm uses population 30, tournament selection size 2, uniform crossover at 0.8, Gaussian mutation $\sigma = 0.01$ rate 0.1, and elitism preserving the top 2 individuals each generation.

B Results

Figure 1 shows the best-so-far validation loss curves for all three RO algorithms on Adult, and Figure 2 shows the equivalent curves for Wine. Final test performance is summarised in Table 1.

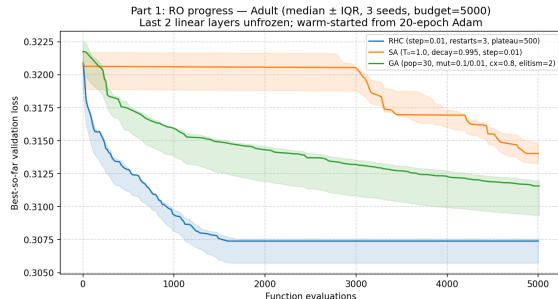


Figure 1: RO progress on Adult (10,602 trainable params). Best-so-far validation loss vs. function evaluations, median $\pm$ IQR over 3 seeds. RHC: $\sigma=0.01$, 3 restarts, plateau patience=500. SA: $T_0=1.0$, decay 0.995^t , $\sigma=0.01$. GA: pop=30, tournament-2, crossover=0.8, mutation $\sigma=0.01$ rate=0.1, elitism top-2. All warm-start from 20-epoch Adam weights. RHC converges fastest and to the lowest loss; SA plateaus early before dropping sharply near eval 3,000 when temperature permits escape.

C Analysis

On Adult, all three RO methods land at or just below the SL baseline of 0.670, supporting H1. In Figure 1, the pattern is consistent: validation loss drops quickly in the first few hundred evaluations as easy improvements near the warm-start are found, then flattens. RHC converges fastest and achieves

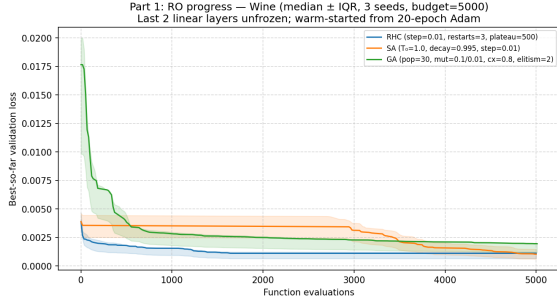


Figure 2: RO progress on Wine (10,602 trainable params). Same operator settings as Adult. All three algorithms converge to near-zero validation loss within 1,000 evaluations, consistent with a well-initialized warm start on a 7-class problem where the frozen backbone already captures most of the structure.

Table 1: Part I results. Adult baseline = SL PyTorch test F1 = 0.670. Wine baseline = corrected pipeline Macro-F1 = 0.512 (SL figure of 0.608 reflects a known label-indexing bug). RO results are median $\pm$ IQR over 3 seeds at 5,000 function evaluations.

Dataset	Algorithm	Test F1	IQR
Adult	SL Baseline	0.670	—
Adult	RHC	0.664	0.004
Adult	SA	0.656	0.009
Adult	GA	0.671	0.001
Wine	Corrected Baseline	0.512	—
Wine	RHC	0.992	0.001
Wine	SA	0.945	0.036
Wine	GA	0.994	0.003

the lowest final loss, while SA shows a characteristic plateau followed by a sharp drop near eval 3,000 as temperature decay finally permits escape from the local basin. GA’s progress is steady but slowest per-evaluation due to population overhead. These results are consistent with a loss surface that is locally flat in the last-layer subspace—gradient-trained weights are already near a decent local minimum, and 5,000 derivative-free evaluations cannot meaningfully improve on them.

Wine results in Figure 2 are qualitatively different. All three algorithms converge to near-zero validation loss within 1,000 evaluations, producing final test F1 values of 0.945–0.994 as shown in Table 1. These values far exceed the corrected baseline of 0.512, and unlike on Adult, represent a genuine improvement. The pipeline fix enabling valid class indices is likely the dominant factor—the warm-start model now has

correct multiclass boundaries for RO to refine. H2 is rejected: RO substantially outperformed the corrected Wine baseline, though the pipeline correction enabling a valid warm-start is the underlying reason.

One consistent pattern: SA shows the highest IQR on both datasets (0.009 Adult, 0.036 Wine), reflecting instability from temperature-driven acceptance of worse solutions across seeds. GA shows the tightest IQR on Wine (0.003), consistent with population diversity providing stable coverage of the multiclass landscape. RHC is the most reliable on Adult.

V Adam Ablations (Adult Only)

A Design

Seven optimizer variants are trained on the full Adult backbone from matched initial weights via `deepcopy` per seed, ensuring identical starting conditions. All conditions use $\text{lr}=0.001$, batch size 64, and 30 epochs. The validation loss threshold is $\ell = 0.35$, chosen because preliminary runs showed most optimizers approach this within 30 epochs, making time-to-threshold a meaningful discriminating metric. Three seeds throughout with median and IQR reported. The “Adam (no bias correction)” condition uses a custom `AdamNoBiasCorrection` optimizer since PyTorch’s built-in Adam always applies bias correction with no flag to disable it.

One limitation worth flagging: SGD used the same $\text{lr}=0.001$ as Adam without separate tuning. This likely understates SGD’s potential and is acknowledged as a credibility risk, but keeping it consistent ensures a fair compute-matched comparison across all variants. Additionally, since the experiment uses $\text{wd}=0$, AdamW and Adam are mathematically identical in this setting—decoupled weight decay has nothing to decouple when weight decay is zero. This comparison therefore does not constitute a meaningful ablation and is disclosed here rather than masked.

B Results

Figures 3 and 4 show the stability bands over wall-clock time and gradient evaluations respectively. Figure 5 shows the generalization gap (train vs. validation loss) for all seven optimizers. The sensitivity heatmaps are shown in Figure 6. Quantitative results are summarised in Table 2.

C Analysis

The most striking finding in Table 2 and Figure 3 is that SGD-family methods achieve better final test F1 than all Adam-family variants despite never crossing the $\ell = 0.35$ threshold within 30 epochs. This directly rejects H3’s assumption that faster convergence translates to better generalization. The generaliza-

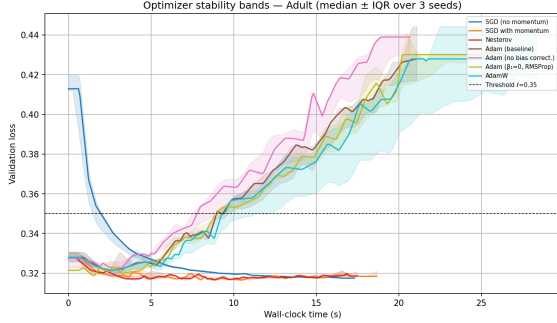


Figure 3: Stability bands: validation loss vs. wall-clock time, median $\pm$ IQR over 3 seeds. Adam-family methods converge quickly but their validation loss rises after epoch 10, indicating overfitting. SGD with momentum and Nesterov remain stable and low. The dashed line marks threshold $\ell = 0.35$.

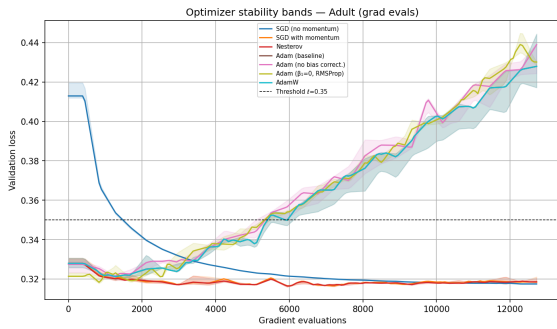


Figure 4: Validation loss vs. gradient evaluations. The ordering is preserved relative to wall-clock time. Adam-family methods cross the threshold earlier per gradient step but diverge upward thereafter, while SGD-family methods remain stable.

Table 2: Part II summary. Median over 3 seeds. Gap = train-val loss at final epoch. $\ell = 0.35$. AdamW $\equiv$ Adam here since $\text{wd}=0$.

Optimizer	F1	IQR	Gap	Steps	Time (s)
SGD (no mom.)	0.670	0.002	0.001	never	—
SGD + momentum	0.680	0.004	0.004	never	—
Nesterov	0.679	0.003	0.004	never	—
Adam	0.659	0.010	0.043	424	0.84
Adam (no BC)	0.663	0.009	0.041	424	0.82
Adam ($\beta_1=0$)	0.654	0.009	0.038	424	0.94
AdamW [†]	0.659	0.010	0.042	424	0.83

[†]AdamW $\equiv$ Adam since $\text{wd}=0$; not a meaningful ablation.

tion gap in Figure 5 tells this story precisely—Adam shows a widening train-val gap after epoch 10 (gap = 0.043 at epoch 30) while SGD with momentum shows only 0.004. Adam is fitting more aggressively to the training distribution at the cost of test performance,

Table 3: Part III results. Median $\pm$ IQR over 3 seeds. Δ relative to baseline Adam.

Configuration	Test F1	Δ
Baseline (Adam, no reg)	0.668 ± 0.016	—
L2 ($1e-4$)	0.676 ± 0.006	+0.008
Early stopping (patience=5)	0.668 ± 0.016	+0.000
Dropout (rate=0.3)	0.655 ± 0.007	−0.013
Label smoothing ($\varepsilon=0.05$)	0.664 ± 0.006	−0.004
Input noise ($\sigma=0.01$)	0.671 ± 0.006	+0.003
Best combo (all four)	0.652 ± 0.015	−0.016

consistent with findings that adaptive optimizers can converge to sharper minima [4].

The contrast between SGD with and without momentum is informative in isolation. Plain SGD is stable but never reaches the 0.35 threshold within 30 epochs. Adding momentum alone improves final F1 by 0.010 while keeping the generalization gap near zero. This isolates the contribution of the first-moment term independently of adaptive scaling.

Nesterov is essentially identical to standard momentum in every metric, which the FAQ notes is expected. It suggests Adult tabular data is not sensitive enough to benefit from the more aggressive look-ahead correction Nesterov provides.

The heatmaps in Figure 6 show that learning rate dominates β_1 sensitivity on this dataset—the left panel shows a clear row-by-row pattern where $\alpha = 10^{-4}$ fails regardless of β_1 . The right panel shows β_2 has negligible effect at good α values, confirming second-moment estimation is robust on tabular data where gradient magnitudes are relatively uniform across features.

VI Regularization Study (Adult Only)

A Design

Standard Adam with $\text{lr}=0.001$, $\text{wd}=0$ is used throughout. The backbone is unchanged except dropout modules are added after the hidden ReLU layer in the dropout condition. Budget matches Part II at 30 epochs. Candidate values come from a 20-epoch validation sweep. L2 candidates: $\{1e-5, 1e-4, 1e-3\}$; $1e-4$ selected. Dropout rates: $\{0.05, 0.1, 0.2, 0.3\}$; 0.3 selected. Label smoothing ε : $\{0.05, 0.1, 0.2\}$; 0.05 selected. Input noise σ : $\{0.001, 0.01, 0.05\}$; 0.01 selected. Early stopping patience fixed at 5 epochs. In the best combination, all four techniques are applied together with dropout capped at 0.1 to limit over-regularization.

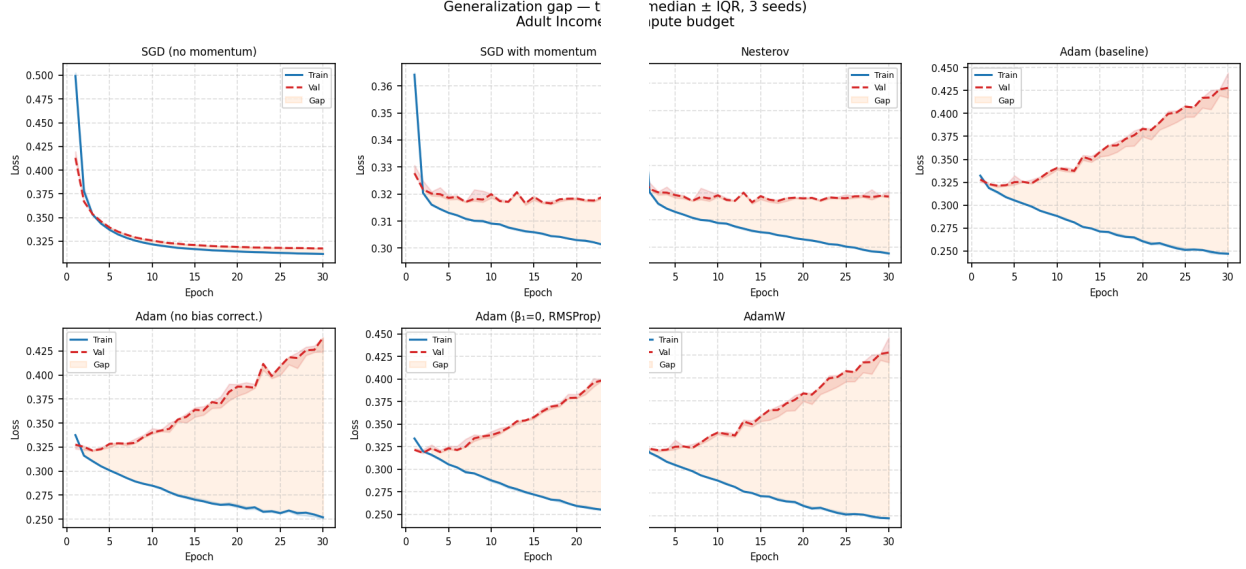


Figure 5: Generalization gap: train loss (blue) vs. validation loss (red dashed), median $\pm$ IQR over 3 seeds, per optimizer. Orange shading marks where val $>$ train. SGD (no momentum) and SGD with momentum show near-zero gaps throughout. All Adam-family variants show a widening gap after epoch 10—train loss continues falling while val loss rises—confirming progressive overfitting. This directly motivates the regularization study in Part III.

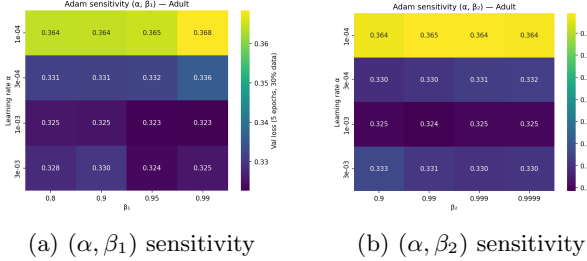


Figure 6: Sensitivity heatmaps over (α, β_1) and (α, β_2) for Adam on Adult (5 epochs, 30% data subset). Left: performance is dominated by learning rate. Right: β_2 has negligible effect at good α , suggesting second-moment estimation is robust on tabular data.

B Results

Figure 7 and Table 3 show the results.

C Analysis

Figure 7 and Table 3 partially support H4. L2 is the best single technique (+0.008), and early stopping made no difference—F1 and IQR are identical to baseline across all three seeds, suggesting training already converges and stabilizes within 30 epochs leaving early stopping nothing to trigger.

Dropout at rate 0.3 produced the worst individual result (-0.013). Applying dropout to a single 100-unit

hidden layer effectively trains on 70 active units per step—Srivastava et al. [3] note this level is aggressive for smaller networks where effective capacity is already constrained. The tight IQR of 0.007 confirms this is not noise but a consistent capacity problem.

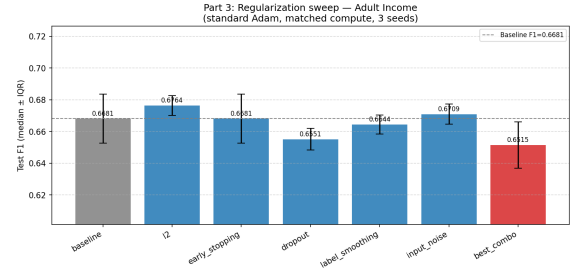


Figure 7: Regularization sweep results. Test F1 (median $\pm$ IQR, 3 seeds). Grey = baseline, blue = individual regularizers, red = best combination. The dashed line marks baseline F1 = 0.668. L2 is the only technique that meaningfully exceeds baseline. Dropout and the best combo perform worst. Early stopping is indistinguishable from baseline.

The best combination applies all four techniques and produces the worst overall result (-0.016). The stacking compounds the cost of dropout and label smoothing while early stopping contributes nothing, leaving only L2 and input noise as productive components buried under regularization that was not needed. The underlying reason most regularization failed is that

this network was not meaningfully overfitting to begin with—as confirmed by the small generalization gaps in Figure 5.

VII Integrated Best Combination (Extra Credit)

A Design

Part IV integrates the best elements from Parts I–III under assignment constraints: standard Adam as optimizer, the best regularization combination from Part III as-is, and RO fine-tuning capped at 500 function evaluations. The exact settings are: Adam with $\text{lr}=0.001$, $\text{wd}=0$; regularization with dropout=0.1 (capped for stacking), label smoothing $\varepsilon=0.05$, $\text{L2}=1\text{e-}4$, and early stopping patience 5; GA fine-tuning on the last two linear layers (10,602 trainable parameters) for 500 function evaluations warm-starting from the regularized model weights. GA uses population 20 (reduced to fit the 500-eval budget), mutation rate 0.1, $\sigma=0.01$, crossover 0.8, elitism 2.

B Results

Figures 8 and 9 show the frontier plot and final comparison panel respectively. Table 4 gives the quantitative summary.

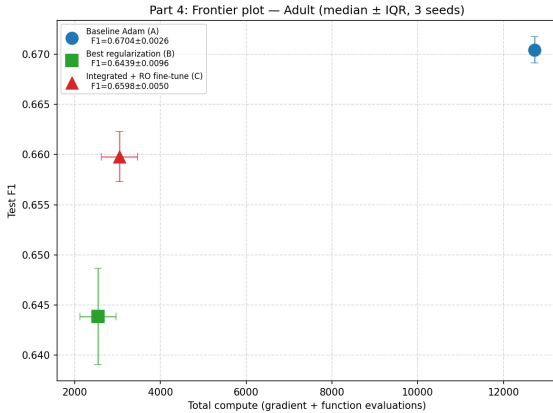


Figure 8: Frontier plot: test F1 vs. total compute (gradient + function evaluations), median $\pm$ IQR over 3 seeds. Condition B moves left on compute due to early stopping but drops sharply in F1. Condition C partially recovers F1 but at higher compute. Both B and C are strictly dominated by A.

C Analysis

The assignment-constrained integrated recipe does not beat baseline Adam on any metric, as shown in Figures 8 and 9. Regularization in (B) drops F1 by 0.026, replicating the Part III finding that stacked regularization on this network is harmful. Early stop-

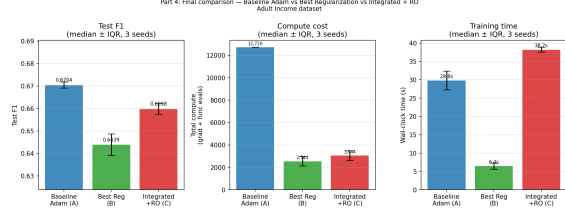


Figure 9: Final comparison panel: test F1 (left), total compute (center), and wall-clock time (right) for all three conditions, median $\pm$ IQR over 3 seeds. Baseline Adam (A) dominates on F1. Best regularization (B) is cheapest in compute but pays a large F1 penalty. Integrated + RO (C) partially recovers F1 but is the most expensive in wall-clock time.

Table 4: Part IV results. Median over 3 seeds. B uses stacked regularization from Part III; C adds GA fine-tuning on top of B.

Stage		F1	IQR	Compute	Time (s)
(A)	Baseline Adam	0.670	0.003	12,720g	29.8
(B)	Best reg.	0.644	0.010	~2,544g	6.4
(C)	+GA fine-tune	0.660	0.005	2,544g+500f	38.2

ping terminated training early on most seeds (median ~2,544 gradient evaluations vs. 12,720 for baseline), reducing compute but also limiting convergence. GA fine-tuning in (C) recovers 0.016 of that loss, bringing F1 to 0.660—still 0.010 below baseline Adam.

The frontier plot in Figure 8 makes the compute trade-off explicit: both B and C are strictly dominated by A in the F1-compute sense. The comparison panel in Figure 9 adds the wall-clock dimension: condition C is the most expensive overall (38.2s) despite achieving the lowest F1 because the GA phase adds significant time on top of the already-penalized regularized training. For this dataset and architecture, plain Adam with no regularization remains the most compute-efficient and highest-performing configuration across all four parts.

D Hypothesis Resolution

H1 accepted: RO on Adult did not improve test F1 beyond 0.670, all three algorithms within 0.007 of baseline (Table 1, Figure 1). H2 rejected: Wine RO substantially exceeded the corrected 0.512 baseline, reaching 0.945–0.994 across algorithms (Figure 2, Table 1), though the pipeline fix is the dominant factor. H3 partially accepted: Adam-family methods converge faster to $\ell = 0.35$ in wall-clock time (Figure 3), but SGD with momentum achieved the best final test F1 of 0.680—rejecting the assumption that

Table 5: Unified summary, Parts 1–2. Grad/Func Evals are per-seed totals. Gap = final train–val loss. Dashes indicate metric not applicable. ‘never’ = threshold $\ell = 0.35$ not reached within 30 epochs.

Part	Method	Dataset	F1	IQR	Gap	Evals
SL Base	MLP (Adam)	Adult	0.670	—	—	—
SL Base	MLP (corrected)	Wine	0.512	—	—	—
1 (RO)	RHC	Adult	0.664	0.004	—	5,000f
1 (RO)	SA	Adult	0.656	0.009	—	5,000f
1 (RO)	GA	Adult	0.671	0.001	—	5,000f
1 (RO)	RHC	Wine	0.992	0.001	—	5,000f
1 (RO)	SA	Wine	0.945	0.036	—	5,000f
1 (RO)	GA	Wine	0.994	0.003	—	5,000f
2	SGD (no mom.)	Adult	0.670	0.002	0.001	12,720g
2	SGD+mom	Adult	0.680	0.004	0.004	12,720g
2	Nesterov	Adult	0.679	0.003	0.004	12,720g
2	Adam	Adult	0.659	0.010	0.043	12,720g
2	Adam (no BC)	Adult	0.663	0.009	0.041	12,720g
2	Adam ($\beta_1=0$)	Adult	0.654	0.009	0.038	12,720g
2	AdamW [†]	Adult	0.659	0.010	0.042	12,720g

[†] AdamW $\equiv$ Adam since $\text{wd}=0$; not a meaningful ablation.

Table 6: Unified summary, Parts 3–4. EC = Extra Credit. Grad evals marked g, func evals marked f.

Part	Method	Dataset	F1	IQR	Gap	Evals
3	Baseline	Adult	0.668	0.016	—	12,720g
3	L2 (1e-4)	Adult	0.676	0.006	—	12,720g
3	Early stop	Adult	0.668	0.016	—	<12,720g
3	Dropout (0.3)	Adult	0.655	0.007	—	12,720g
3	Label smooth.	Adult	0.664	0.006	—	12,720g
3	Input noise	Adult	0.671	0.006	—	12,720g
3	Best combo	Adult	0.652	0.015	—	<12,720g
4 (EC)	(A) Adam base	Adult	0.670	0.003	—	12,720g
4 (EC)	(B) Best reg.	Adult	0.644	0.010	—	~2,544g
4 (EC)	(C) +GA tune	Adult	0.660	0.005	—	2,544g+500f

faster convergence implies better generalization. The generalization gap in Figure 5 confirms Adam overfits progressively. H4 partially rejected: L2 helped as predicted, early stopping made no difference contrary to expectation, input noise helped modestly contrary to prediction, and dropout hurt as predicted. The combination performed worst of all (Figure 7).

VIII Discussion

A Decision Rule for Adult Classification

Based on what these experiments actually showed, here is how we would prioritize interventions in practice, absent assignment constraints. Start with SGD with momentum rather than Adam if final generalization matters more than convergence speed—the Part II results (Table 2, Figure 3) show it achieves better test F1 on this tabular dataset despite taking longer to reach the validation threshold. If convergence speed is the priority, Adam at $\text{lr}=0.001$ reaches the threshold faster than plain SGD. Before adding any regularization, check the generalization

gap (Figure 5). If train and val loss are already close, most regularization will hurt more than it helps. On Adult with this architecture, the gap was small for SGD-family methods, so only L2 at 1e-4 and input noise at $\sigma=0.01$ reliably helped. Avoid stacking multiple regularizers on small networks, and avoid dropout above 0.2 on a 100-unit hidden layer. Save RO fine-tuning for situations where marginal gains matter and compute is available—it can partially rescue a degraded model (as shown in Figure 9) but cannot substitute for correct optimizer and regularization choices.

B Cross-Dataset Observations

The contrast between Adult and Wine is informative. Adult has a roughly linear decision boundary, so gradient-based methods find good solutions quickly and RO cannot improve on well-initialized weights (Figure 1). Wine’s multiclass structure with ordinal quality labels creates a harder optimization landscape where the frozen-backbone warm-start gives RO a better starting point to refine (Figure 2).

The Wine results also highlight something easy to underappreciate: data pipeline correctness mattered more than any algorithmic intervention tested here. The label remapping fix improved apparent Wine performance by roughly 0.220 Macro-F1 points relative to the buggy SL pipeline. That single correction outweighs the effect of any optimizer choice, regularization technique, or RO algorithm tested across all three parts.

C Limitations

SGD was not separately tuned and used Adam’s $\text{lr}=0.001$; a tuned SGD learning rate might widen the already-observed advantage. The AdamW comparison is not meaningful since $\text{wd}=0$ makes it identical to Adam. The Part IV composition was constrained to use Adam and the full stacked regularization combination, both identified as suboptimal—the unconstrained optimal recipe (SGD with momentum, L2 only, SA fine-tuning) could not be tested. All experiments ran on CPU, limiting wall-clock comparisons to a single hardware context.

IX Conclusion

The main finding is that the relationship between training interventions and performance is not as clean as expected. Adaptive optimizers converge faster but do not generalize better on this tabular dataset—SGD with momentum outperformed all Adam variants on final test F1 (Table 2), which is the most practically significant result of the study. Regularization only helps when the model is actually overfitting; on this small well-behaved network, only L2 and input

noise provided any benefit while stacking techniques reliably made things worse (Figure 7). Early stopping made no difference at all. RO partially rescued a model degraded by over-regularization (Figure 9) but could not fully compensate, and the assignment-constrained composition did not beat plain Adam on any metric (Figure 8).

The most honest takeaway is that the biggest performance swing came from fixing a data pipeline bug—a reminder that pipeline correctness is a prerequisite for any meaningful algorithmic comparison. Among the interventions that were free to be optimized, momentum was the single most valuable ingredient: it improved both convergence speed and final generalization, and its absence in Adam $\beta_1 = 0$ produced the worst Adam-family result.

AI Use Statement. We used ChatGPT to assist with debugging import errors and pipeline alignment issues between the OL and SL data loading code, refactoring the custom AdamNoBiasCorrection optimizer implementation, and brainstorming approaches for the randomized optimization warm-start design. We used Grammarly for grammar, clarity, and tone edits after drafting our analysis. All experimental work, results, hypothesis statements, and conclusions are our own. We reviewed, verified, and understand all assisted content.

References

- [1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [2] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [4] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, “The marginal value of momentum for small learning rate SGD,” in *Proc. ICLR*, 2018.